

Visualização em Tempo Real

Apresentação Trabalho Prático

Ivan Ribeiro, Francisco Ferreira e Júlio Pinto

Simulador de partículas determinístico com GPU

- Rust
- WGPU

Segurança, performance, controlo sobre a pipeline de renderização, APIs modernas

- Grande escala
- Determinismo
- Colisões

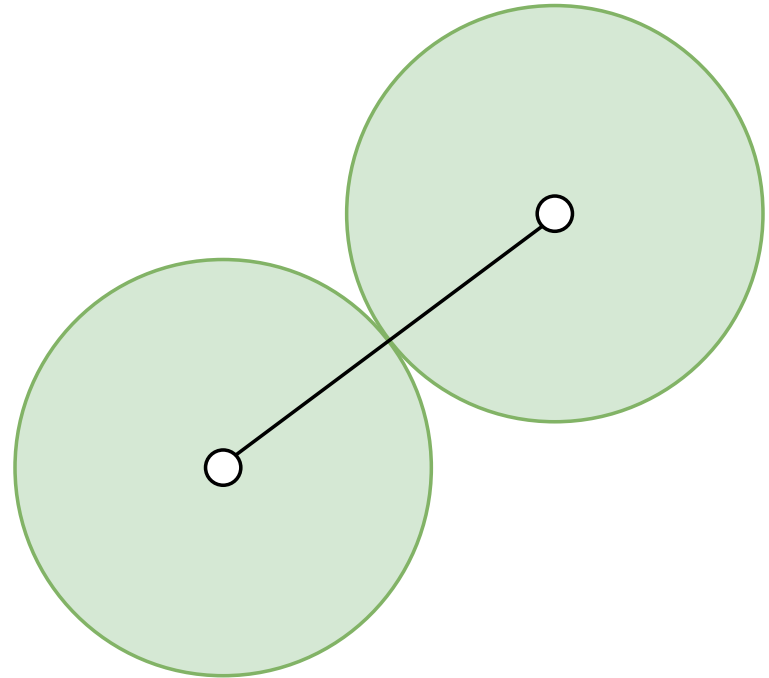
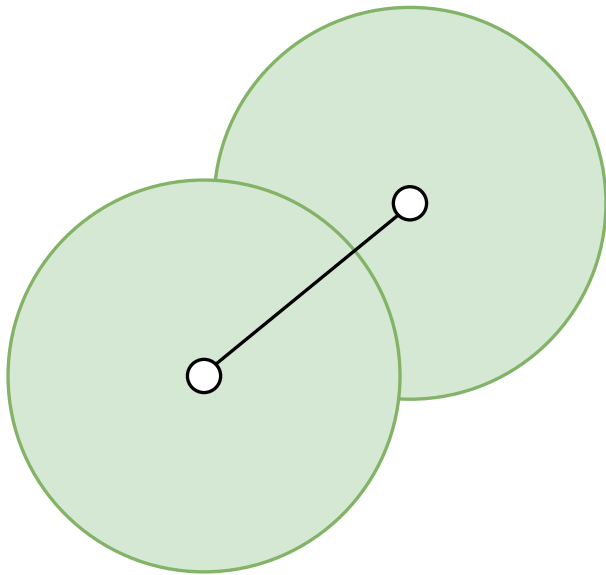
Que equações usar para mover as partículas? E para as colidir?

- **Basic Verlet Integration**
- Velocity Verlet
- Euler integration
- Leapfrog Integration
- Position Verlet
- Runge-Kutta Methods (e.g., RK4)
- Backward Euler

```
struct ParticlePhysics {  
    pos: Vec2,  
    old_pos: Vec2,  
    accel: Vec2,  
}
```

Em cada passo, dado um Δt :

```
let vel = particle.pos - particle.old_pos;  
particle.old_pos = particle.pos;  
let accel = particle.accel;  
particle.pos += vel + (accel * DELTA_SQUARED);  
particle.accel = Vec2::ZERO;
```



Otimizações

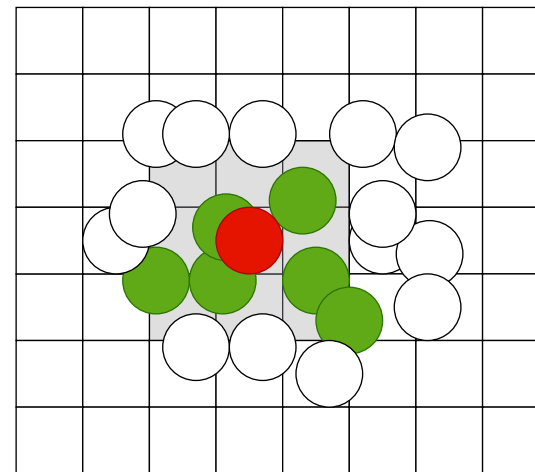
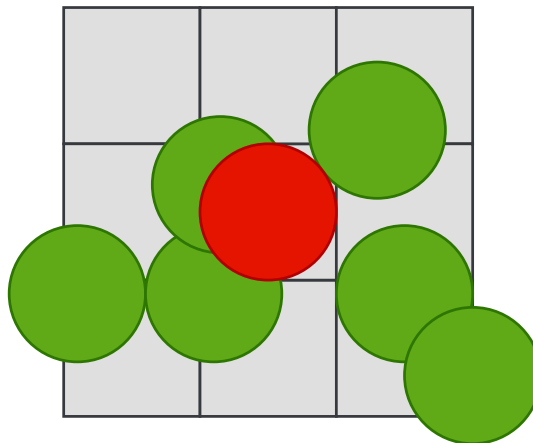
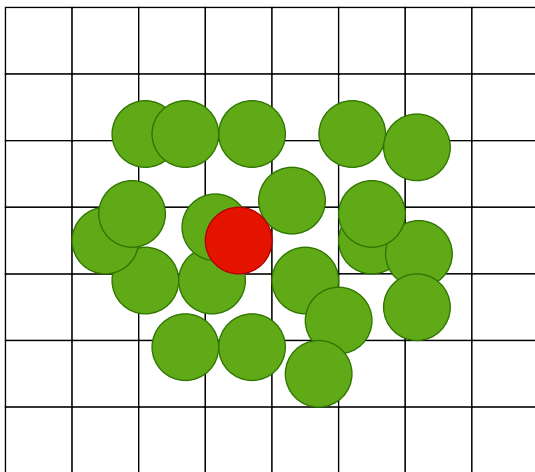
Atual:

- Comparar todas as particulas com todas as outras (N^2)
- Nao paralelizável
- **10k** particulas: **4FPS**

Objetivo:

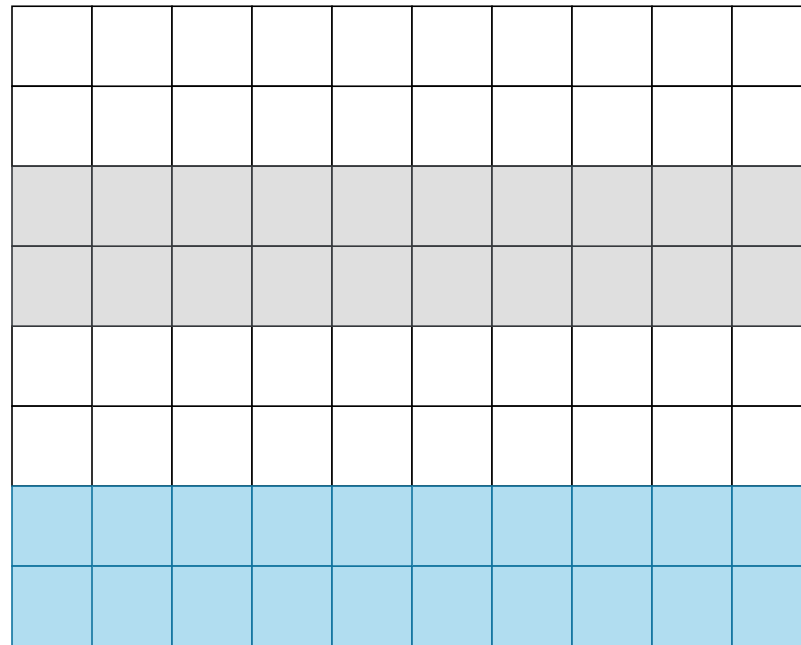
- **100k** particulas, **60FPS**

- Dividir o espaço em grelha
- Comparar células adjacentes



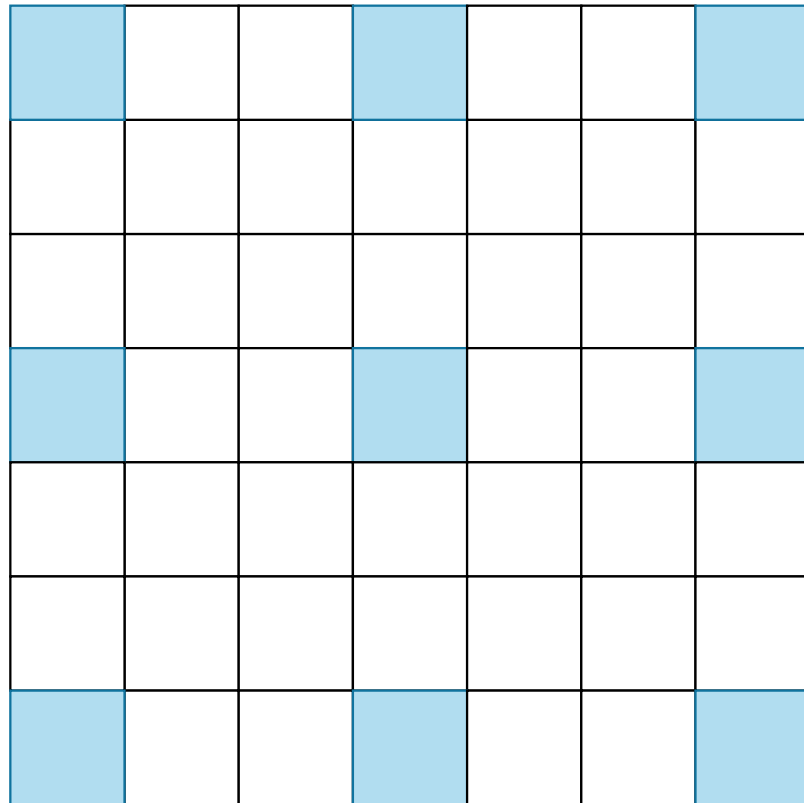
- **10k** partículas: **300FPS**

- [illegible]

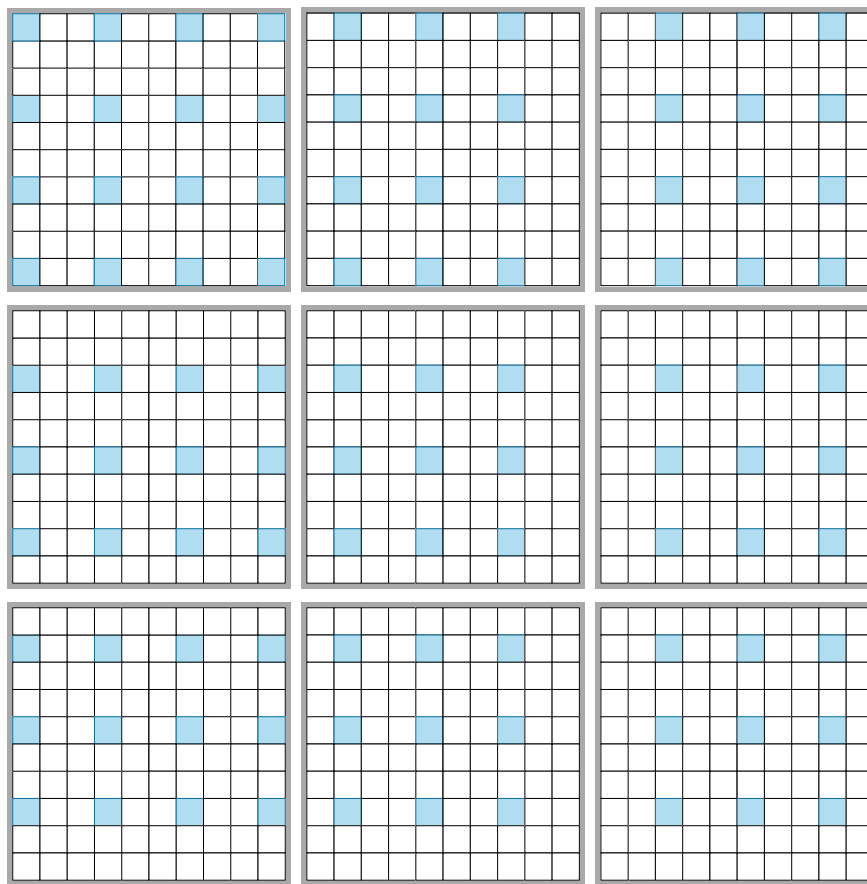


- **50k** particulas: **50FPS**

- Não podemos usar a mesma estrutura do multithreading
- Como manter determinismo?



- Sao necessarios 9 passes
- **100k** particulas: **120FPS**



Renderização de imagens

- Determinismo permite que partículas acabem todas na mesma posicao

